

IntuiUX Agent

Руководство по установке и развёртыванию

AI- powered UX Research Pipeline

Версия: 1.0.0

Дата: 15.09.2026

Репозиторий: github.com/Wasp1977/intuiux_092026

Развёртывание: intuiux092026.vercel.app

Содержание

1. Общий обзор проекта	5
1.1. Конвейер обработки	5
1.2. Технологический стек	6
2. Структура проекта	6
2.1. Корневые конфигурационные файлы	6
2.2. Исходный код (src/)	7
2.3. Дополнительные каталоги	8
3. Конфигурация LLM API	8
3.1. Файл .z-ai-config	8
3.2. Пути поиска конфигурации	8
3.3. Переменные окружения	9
3.4. Класс LLM (src/lib/zai.ts)	9
3.5. Контрольный список для продакшена	10
4. База данных и Prisma	10
4.1. Конфигурация подключения	10
4.2. Модели данных	10
4.3. PipelineSession и PipelineStage	11
4.4. Команды Prisma	12
5. API-маршруты	12
5.1. Маршрут /api/pipeline	12
5.2. Маршруты этапов конвейера	13
5.3. Прочие API-маршруты	13
6. Фронтенд-архитектура	14
6.1. Главный компонент (page.tsx)	14
6.2. MessageContent.tsx	14
6.3. MermaidRenderer.tsx	15
6.4. UI-компоненты (shadcn/ui)	15
6.5. Стилизация и дизайн-система	15

7. Конфигурация пайплайна	15
7.1. Типы продуктов (ProductType)	16
7.2. Алгоритм detectProductType()	16
7.3. Наборы этапов по типу продукта	17
8. Развёртывание	17
8.1. Локальная разработка	17
8.2. Развёртывание на Vercel	17
8.3. Развёртывание на собственном сервере	18
8.4. Caddyfile (обратный прокси)	18
8.5. Systemd-сервис	19
9. Конфигурация Next.js и TypeScript	19
9.1. next.config.ts	19
9.2. tsconfig.json	20
9.3. Скрипты package.json	20
10. Система анализа конкурентов	20
10.1. idea-extractor.ts (v11.0)	21
10.2. context-aware-competitor-search.ts	21
10.3. competitor-analyzer.ts (v3.0)	21
10.4. Резервная база данных	21
11. Генерация артефактов	22
11.1. Этап Idea (Анализ идеи)	22
11.2. Этап CJM (Карта пути клиента)	22
11.3. Этап Prototype (Прототип)	23
11.4. Mermaid-диаграммы	23
12. Безопасность	24
12.1. Защита учётных данных	24
12.2. Аутентификация API-запросов	24
12.3. Изоляция контента (iframe sandbox)	24
12.4. Обработка ошибок	24
12.5. Остановка генерации	25
13. Устранение неполадок	25

13.1. Логи́рование	25
13.2. Команды проверки конфигурации	26
14. Расширение проекта	26
14.1. Добавление нового этапа конвейера	26
14.2. Добавление нового типа продукта	27
14.3. Замена LLM-провайдера	27

1. Общий обзор проекта

IntuiUX Agent представляет собой современное веб-приложение, построенное на базе Next.js 16, которое реализует полностью автоматизированный конвейер UX-исследований с использованием искусственного интеллекта. Основная цель проекта - преобразование текстового описания продуктовой идеи в полный набор UX-артефактов, включая карты пути клиента, информационную архитектуру, пользовательские потоки, интерактивные прототипы и метрики оценки. Система спроектирована как единая платформа для исследователей UX, продуктовых аналитиков и дизайнеров, позволяющая им получать профессиональные результаты без необходимости ручного создания каждого артефакта отдельно.

Проект развёрнут на платформе Vercel по адресу <https://intuiux092026.vercel.app/> и доступен для публичного использования. Исходный код размещён в репозитории https://github.com/Wasp1977/intuiux_092026.git. Приложение работает в режиме серверного рендеринга (SSR) с поддержкой клиентской интерактивности через директиву "use client" в главном компоненте страницы. В качестве среды выполнения используется Bun - высокопроизводительный JavaScript-рантайм, обеспечивающий быструю установку зависимостей и сборку проекта.

1.1. Конвейер обработки

Центральной частью системы является 9-этапный конвейер обработки, который последовательно трансформирует исходную идею в набор конкретных UX-артефактов. Каждый этап получает результаты предыдущих этапов как контекст и генерирует собственный выходной артефакт. Конвейер адаптивен: на основе типа продукта (электронная коммерция, SaaS, B2B и др.) система автоматически определяет набор применимых этапов и их порядок выполнения. Это позволяет оптимизировать процесс для различных предметных областей и исключить генерацию нерелевантных артефактов.

Этап	Имя	Название (рус.)	Описание
1	Idea	Идея	Анализ и формализация продуктовой идеи
2	Competitors	Конкуренты	Исследование конкурентного окружения
3	CJM	Карта пути клиента	Визуализация клиентского пути (Mermaid)
4	IA	Информ. архитектура	Структура навигации и ER-диаграмма
5	Userflow	Пользовательский поток	Пошаговый поток взаимодействия
6	Prototype	Прототип	Интерактивный HTML-прототип (SPA)
7	Invitation	Приглашение	Шаблон приглашения к тестированию
8	Guidelines	Руководства	Дизайн-руководства и принципы
9	Metrics	Метрики	Ключевые метрики оценки UX

1.2. Технологический стек

Проект использует современный технологический стек, обеспечивающий высокую производительность, типобезопасность и отличную совместимость компонентов. Ниже представлена подробная таблица всех ключевых технологий и их ролей в проекте.

Технология	Версия	Назначение
Next.js	16.1.1	Фреймворк приложения, SSR, API-маршруты
React	19	Библиотека UI, компонентная модель
TypeScript	5.x	Статическая типизация, безопасность кода
Tailwind CSS	4	Утилитарный CSS-фреймворк
Prisma	6.x	ORM для работы с SQLite базой данных
SQLite	3.x	Встроенная реляционная база данных
z-ai-web-dev-sdk	latest	SDK для доступа к LLM API
Mermaid.js	11.12.3	Генерация диаграмм из текстового описания
shadcn/ui + Radix	latest	Компоненты UI с доступностью
react-markdown	9.x	Рендеринг Markdown-контента
framer-motion	11.x	Анимации и переходы
Bun	1.x	JavaScript-рантайм и пакетный менеджер
Lucide Icons	latest	Набор иконок для интерфейса

2. Структура проекта

Проект следует стандартной структуре Next.js App Router с чётким разделением файлов по функциональному назначению. Корневой каталог содержит конфигурационные файлы сборки и развёртывания, а исходный код организован в каталоге `src/` с подкаталогами `app/`, `components/`, `lib/` и `hooks/`. API-маршруты сгруппированы по доменной области в каталоге `api/`, а каждый этап конвейера имеет собственный маршрут в подпапке `pipeline/stages/`. Такое разделение обеспечивает удобство навигации по кодовой базе и независимость модулей друг от друга.

2.1. Корневые конфигурационные файлы

```
intuiux-agent/  
  package.json           # Зависимости и скрипты проекта  
  next.config.ts         # Конфигурация Next.js (standalone, ignoreBuildErrors)  
  tsconfig.json          # Конфигурация TypeScript (ES2017, bundler)  
  vercel.json            # Настройки развёртывания Vercel  
  prisma/  
    schema.prisma        # Схема базы данных Prisma (8 моделей)  
  Caddyfile              # Конфигурация обратного прокси Caddy  
  .env.example           # Шаблон переменных окружения  
  .gitignore             # Исключения Git (включая .z-ai-config)  
  .dockerignore          # Исключения Docker
```

```

components.json      # Конфигурация shadcn/ui
tailwind.config.ts   # Конфигурация Tailwind CSS
postcss.config.mjs   # Конфигурация PostCSS
eslint.config.mjs     # Конфигурация ESLint

```

2.2. Исходный код (src/)

```

src/
  app/
    page.tsx          # Главный UI-компонент (1875 строк, use client)
    layout.tsx         # Корневой макет страницы (шрифты Geist)
    globals.css        # Глобальные стили (Tailwind, тёмная тема)
  api/
    chat/route.ts      # API чата с LLM
    pipeline/
      route.ts         # CRUD сессий конвейера
    stages/
      idea/route.ts    # Этап 1: Анализ идеи
      competitors/route.ts # Этап 2: Конкурентный анализ
      cjm/route.ts     # Этап 3: Карта пути клиента
      ia/route.ts      # Этап 4: Информационная архитектура
      userflow/route.ts # Этап 5: Пользовательский поток
      prototype/route.ts # Этап 6: Прототип
      invitation/route.ts # Этап 7: Приглашение
      guidelines/route.ts # Этап 8: Руководства
      metrics/route.ts # Этап 9: Метрики
    prototype/
      route.ts         # CRUD прототипов
      [slug]/route.ts  # Доступ к прототипу по слаггу
    knowledge/route.ts  # API базы знаний
    insights/route.ts  # API инсайтов
    metrics/route.ts   # API метрик
    agents/route.ts    # API агентов
    deploy/route.ts    # API развёртывания
    vercel-deploy/route.ts # API развёртывания Vercel
    github/route.ts    # API интеграции GitHub
    competitors-search/route.ts # API поиска конкурентов
    demo/route.ts     # Демо-данные
  components/
    MessageContent.tsx # Рендеринг сообщений (iframe, Mermaid, Markdown)
    MermaidRenderer.tsx # Компонент рендеринга диаграмм Mermaid
    ui/                # 50+ компонентов shadcn/ui
    (button, card, dialog, tabs, table, select, input, ...)
  lib/
    zai.ts             # Класс LLM (Singleton, chat(), X-Token)
    pipeline-config.ts # Конфигурация конвейера (ProductType, этапы)
    db.ts              # Prisma Client (Singleton pattern)
    idea-extractor.ts  # Извлечение идеи (v1.0)
    competitor-analyzer.ts # Анализ конкурентов (v3.0)
    context-aware-competitor-search.ts # Контекстный поиск конкурентов
    cjm-generator.ts   # Генератор CJM (Mermaid journey)
    prototype-generator.ts # Генератор HTML-прототипов
    agents-fallback.ts # Резервные данные агентов
    agent-context.ts   # Контекст агента
    github-pages.ts    # Публикация на GitHub Pages
    vercel-deploy.ts   # Развёртывание на Vercel
    demo-data.ts       # Демо-данные для тестирования
    utils.ts           # Утилиты (сп - объединение классов)
  hooks/
    use-toast.ts       # Хук уведомлений

```

2.3. Дополнительные каталоги

Помимо основного исходного кода, проект содержит каталог `prototypes/` для сохранения сгенерированных HTML-прототипов, каталог `docs/` с документацией и сгенерированными страницами, а также каталог `db/` с файлом базы данных SQLite (`custom.db`). Каталог `scripts/` содержит вспомогательные скрипты TypeScript для инициализации агентов, создания данных и обновления компонентов системы. Эти скрипты используются преимущественно в процессе разработки и не являются частью рабочего приложения.

3. Конфигурация LLM API

Система IntuiUX Agent использует большую языковую модель (LLM) через `z-ai-web-dev-sdk` для генерации всех UX-артефактов. Конфигурация подключения к API осуществляется через файл `.z-ai-config`, который содержит базовый URL и ключ API. Этот файл является основным источником учётных данных для взаимодействия с LLM-провайдером. Система также поддерживает переопределение через переменные окружения, что особенно полезно при развёртывании на платформах вроде Vercel, где файловая конфигурация недоступна.

3.1. Файл .z-ai-config

Конфигурационный файл `.z-ai-config` представляет собой JSON-документ с двумя обязательными полями: `baseUrl` и `apiKey`. Файл ищется в нескольких каталогах в порядке приоритета: текущий рабочий каталог (`cwd`), домашний каталог пользователя (`~`), системный каталог (`/etc`) и временный каталог (`/tmp`). Такой порядок поиска обеспечивает гибкость конфигурации при различных сценариях использования - от локальной разработки до контейнерного развёртывания.

```
// .z-ai-config (JSON)
{
  "baseUrl": "https://api.llm-provider.com/v1",
  "apiKey": "sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxx"
}
```

3.2. Пути поиска конфигурации

Приоритет	Путь	Описание
1	<code>process.cwd()/.z-ai-config</code>	Наивысший приоритет, локальная разработка
2	<code>~/.z-ai-config</code>	Домашний каталог пользователя
3	<code>/etc/.z-ai-config</code>	Системная конфигурация (Linux)
4	<code>/tmp/.z-ai-config</code>	Временная конфигурация (контейнеры)

3.3. Переменные окружения

В дополнение к файлу конфигурации, система поддерживает ряд переменных окружения, которые могут переопределять значения из `.z-ai-config` или предоставлять дополнительные настройки. Переменные окружения имеют более высокий приоритет, чем файл конфигурации, что позволяет гибко управлять поведением системы при развёртывании без изменения файлов.

Переменная	Описание	Пример
ZAI_BASE_URL	URL API LLM-провайдера	https://api.llm.com/v1
ZAI_API_KEY	Ключ аутентификации API	sk-xxxx...
DATABASE_URL	URL подключения к БД SQLite	file:./dev.db
GITHUB_TOKEN	Токен GitHub API	ghp_xxxx...
SEARCH_API_KEY	Ключ API поиска конкурентов	search-xxxx...

3.4. Класс LLM (src/lib/zai.ts)

Модуль `zai.ts` экспортирует класс LLM, реализованный по шаблону Singleton, что гарантирует единственный экземпляр подключения на протяжении жизненного цикла приложения. Класс предоставляет метод `chat()`, который выполняет HTTP-POST запрос к эндпоинту `${baseUrl}/chat/completions` с заголовком X-Token для аутентификации. Запрос содержит модель, сообщения (`system + user`), параметры температуры и максимального количества токенов. Ответ парсится как JSON и возвращается в виде строки контента. При возникновении ошибок выполняется логирование и выбрасывается исключение с детальным описанием.

```
// src/lib/zai.ts (упрощённо)
class LLM {
  private static instance: LLM;
  private baseUrl: string;
  private apiKey: string;

  private constructor() {
    // Чтение из .z-ai-config или переменных окружения
    const config = this.loadConfig();
    this.baseUrl = config.baseUrl;
    this.apiKey = config.apiKey;
  }

  static getInstance(): LLM {
    if (!LLM.instance) LLM.instance = new LLM();
    return LLM.instance;
  }

  async chat(messages: Message[], opts?: ChatOpts): Promise<string> {
    const resp = await fetch(`${this.baseUrl}/chat/completions`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',

```

```

    'X-Token': this.apiKey,
  },
  body: JSON.stringify({
    model: opts?.model || 'default',
    messages,
    temperature: opts?.temperature ?? 0.7,
    max_tokens: opts?.maxTokens ?? 4096,
  }),
});
const data = await resp.json();
return data.choices[0].message.content;
}
}

```

3.5. Контрольный список для продакшена

- Создать файл `.z-ai-config` с корректными `baseUrl` и `apiKey`
- Убедиться, что `.z-ai-config` добавлен в `.gitignore` (не коммитить ключи)
- При развёртывании на Vercel установить переменные `ZAI_BASE_URL` и `ZAI_API_KEY`
- Проверить доступность API-эндпоинта: `curl -H "X-Token: $KEY" $BASE_URL/models`
- Убедиться, что `DATABASE_URL` указывает на доступный файл SQLite
- При необходимости установить `GITHUB_TOKEN` для публикации прототипов

4. База данных и Prisma

Проект использует SQLite в качестве базы данных с Prisma ORM в качестве слоя абстракции. SQLite выбран за счёт своей простоты, отсутствия необходимости в отдельном сервере базы данных и полной совместимости с моделью развёртывания на Vercel (serverless). Подключение к базе данных осуществляется через переменную окружения `DATABASE_URL`, которая по умолчанию указывает на локальный файл `dev.db`. Prisma Client создаётся как Singleton для предотвращения множественных подключений при горячей перезагрузке в режиме разработки, что является стандартной практикой для Next.js-приложений.

4.1. Конфигурация подключения

```

// .env
DATABASE_URL="file:./dev.db"

// src/lib/db.ts (Singleton Prisma Client)
import { PrismaClient } from '@prisma/client';

const globalForPrisma = globalThis as unknown as { prisma: PrismaClient };
export const prisma = globalForPrisma.prisma || new PrismaClient();
if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma;
export default prisma;

```

4.2. Модели данных

Схема базы данных содержит 8 основных моделей, каждая из которых обслуживает определённую функциональную область приложения. Модели связаны между собой через внешние ключи и отношения, обеспечивая целостность данных. Ниже представлено описание каждой модели с её ключевыми полями и назначением.

Модель	Назначение	Ключевые поля
Agent	Конфигурация AI-агентов (роль, промпт, модель)	id, name, role, systemPrompt, model
KnowledgeBase	База знаний для контекста агентов	id, agentId, content, embedding
Conversation	Пользовательские диалоги	id, title, createdAt, updatedAt
Message	Сообщения в диалогах	id, conversationId, role, content
Insight	UX-инсайты из исследований	id, category, content, priority
Metric	Метрики оценки UX	id, name, value, unit, sessionId
UserSession	Сессии пользователей	id, userId, data, expiresAt
PrototypeDeployment	Развёрнутые прототипы	id, slug, url, html, sessionId

4.3. PipelineSession и PipelineStage

Две важнейшие модели для работы конвейера - PipelineSession и PipelineStage. PipelineSession представляет собой полный сеанс обработки продуктовой идеи и содержит общую информацию о сессии: идентификатор, входную идею, определённый тип продукта, текущий статус обработки (pending, processing, completed, error), а также итоговый результат в формате JSON. Каждая сессия может содержать несколько этапов PipelineStage, которые отслеживают индивидуальный прогресс каждого шага конвейера.

PipelineStage хранит детальную информацию о каждом этапе: имя этапа (idea, competitors, cjm и т.д.), статус (pending, processing, completed, skipped, error), результат обработки в формате JSON, текстовое описание результата, временные метки начала и завершения, а также сообщения об ошибках. Отношение между PipelineSession и PipelineStage - "один ко многим", что позволяет запросить все этапы конкретной сессии для отображения прогресса в пользовательском интерфейсе.

```
// prisma/schema.prisma (фрагмент)
model PipelineSession {
  id          String      @id @default(cuid())
  idea        String
  productType String?
  status      String      @default("pending")
  result      Json?
  stages      PipelineStage[]
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
}

model PipelineStage {
  id          String      @id @default(cuid())
  sessionId   String
  session     PipelineSession @relation(fields: [sessionId], references: [id])
}
```

```
stageName String
status String @default("pending")
result Json?
resultText String?
startedAt DateTime?
completedAt DateTime?
error String?
}
```

4.4. Команды Prisma

Команда	Описание
bunx prisma generate	Генерация Prisma Client из схемы
bunx prisma db push	Применение схемы к БД без миграций
bunx prisma migrate dev	Создание и применение миграций (dev)
bunx prisma migrate deploy	Применение миграций (production)
bunx prisma db reset	Сброс базы данных (удаление всех данных)
bunx prisma studio	Запуск визуального редактора БД

5. API-маршруты

API-маршруты являются серверной частью приложения и реализуют всю бизнес-логику обработки запросов. Next.js App Router предоставляет модель маршрутных обработчиков (Route Handlers), где каждый файл route.ts в каталоге api/ экспортирует HTTP-методы (GET, POST, PUT, DELETE). Основной маршрут /api/pipeline управляет сеансами конвейера, а вложенные маршруты /api/pipeline/stages/[stage]/ обрабатывают индивидуальные этапы. Каждый маршрут следует единому паттерну обработки: извлечение параметров, проверка зависимостей, вызов LLM, парсинг результата, сохранение в базу данных.

5.1. Маршрут /api/pipeline

Основной маршрут конвейера поддерживает несколько операций через POST-запросы с различными значениями поля action. Операция создания сессии (action: "create") принимает текст идеи и инициализирует новую PipelineSession с определением типа продукта. Операция получения сессии (action: "getSession") возвращает текущее состояние всех этапов. Операция detect_product_type определяет тип продукта на основе текста идеи с использованием алгоритма оценки ключевых слов из pipeline-config.ts. Операция stop_generation позволяет прервать текущий процесс генерации, устанавливая флаг остановки в сессии.

Метод	Action	Описание	Параметры
POST	create	Создание новой сессии конвейера	{ idea: string }
POST	getSession	Получение текущей сессии и этапов	{ sessionId: string }
POST	detect_product_type	Определение типа продукта	{ idea: string }
POST	stop_generation	Остановка генерации	{ sessionId: string }
GET	-	Список всех сессий	-

5.2. Маршруты этапов конвейера

Каждый этап конвейера реализован как отдельный API-маршрут по адресу `/api/pipeline/stages/[stageName]/route.ts`. Все маршруты этапов следуют единому паттерну обработки, что обеспечивает консистентность и предсказуемость поведения системы. Паттерн включает следующие шаги: извлечение идентификатора сессии из тела запроса, поиск сессии в базе данных, проверка завершённости предыдущих этапов (зависимости), установка статуса "processing" для текущего этапа, формирование промпта с контекстом предыдущих этапов, вызов LLM через класс `zai.ts`, парсинг JSON-ответа, сохранение результата в `PipelineStage`, обновление статуса на "completed". При возникновении ошибки на любом шаге статус устанавливается в "error" с сохранением сообщения об ошибке.

Маршрут	Метод	Описание
<code>/api/pipeline/stages/idea</code>	POST	Анализ идеи, формализация продукта
<code>/api/pipeline/stages/competitors</code>	POST	Поиск и анализ конкурентов
<code>/api/pipeline/stages/cjm</code>	POST	Генерация карты пути клиента (Mermaid)
<code>/api/pipeline/stages/ia</code>	POST	Генерация информационной архитектуры
<code>/api/pipeline/stages/userflow</code>	POST	Генерация пользовательского потока
<code>/api/pipeline/stages/prototype</code>	POST	Генерация HTML-прототипа
<code>/api/pipeline/stages/invitation</code>	POST	Генерация приглашения к тестированию
<code>/api/pipeline/stages/guidelines</code>	POST	Генерация дизайн-руководств
<code>/api/pipeline/stages/metrics</code>	POST	Генерация метрик оценки UX

5.3. Прочие API-маршруты

Маршрут	Метод	Описание
<code>/api/chat</code>	POST	Чат с LLM (прямое взаимодействие)

Маршрут	Метод	Описание
/api/prototype	GET/POST	CRUD прототипов
/api/prototype/[slug]	GET	Получение прототипа по слягу
/api/knowledge	GET/POST	Управление базой знаний
/api/insights	GET	Получение UX-инсайтов
/api/metrics	GET	Получение метрик
/api/agents	GET	Список AI-агентов
/api/deploy	POST	Развёртывание прототипа
/api/vercel-deploy	POST	Развёртывание на Vercel
/api/github	GET/POST	Интеграция с GitHub API
/api/competitors-search	POST	Поиск конкурентов через API
/api/demo	GET	Получение демо-данных

6. Фронтенд-архитектура

Фронтенд приложения построен как одностраничный интерфейс (SPA) в рамках Next.js App Router. Главный компонент страницы (page.tsx) содержит approximately 1875 строк кода и реализован с директивой "use client", что обеспечивает полную клиентскую интерактивность. Интерфейс включает панель ввода идеи, визуальный прогресс-бар конвейера с 9 этапами, область отображения результатов каждого этапа, а также средства навигации между сессиями. Все состояния управляются через React hooks (useState, useEffect, useCallback), а для анимаций используется framer-motion.

6.1. Главный компонент (page.tsx)

Компонент page.tsx является центральной точкой фронтенд-архитектуры. Он управляет полным жизненным циклом сессии конвейера: от ввода пользовательской идеи до отображения финальных артефактов. Компонент содержит состояние текущей сессии, массив этапов с их статусами, результатами и ошибками, а также флаги загрузки и генерации. При отправке идеи компонент последовательно вызывает API-маршруты каждого этапа, обновляя прогресс в реальном времени. Результаты этапов отображаются в соответствующих вкладках или секциях с использованием компонентов MessageContent и MermaidRenderer.

6.2. MessageContent.tsx

Компонент MessageContent отвечает за рендеринг контента сообщений и результатов этапов. Он поддерживает три формата отображения: Mermaid-диаграммы (определяются по блокам ``mermaid`), Markdown-контент (через react-markdown) и HTML-контент

(отображается в изолированном `iframe` с атрибутом `sandbox` для безопасности). При обнаружении блока Mermaid в тексте, компонент делегирует рендеринг компоненту MermaidRenderer. HTML-контент прототипов отображается в `iframe`, что предотвращает конфликты стилей между основным приложением и сгенерированным прототипом. Markdown-контент рендерится с поддержкой заголовков, списков, таблиц и выделения кода.

6.3. MermaidRenderer.tsx

Компонент MermaidRenderer обеспечивает визуализацию диаграмм Mermaid.js непосредственно в браузере. Он использует библиотеку mermaid версии 11.12.3 с конфигурацией тёмной темы (`theme: "dark"`). Компонент принимает текстовое определение диаграммы, вызывает `mermaid.render()` для генерации SVG и вставляет результат в DOM. Обработка ошибок включает перехват исключений парсинга Mermaid и отображение исходного текста в случае неудачи. Компонент поддерживает все типы диаграмм, используемые в проекте: journey (CJM), flowchart (IA, Userflow) и er (ER-диаграммы).

6.4. UI-компоненты (shadcn/ui)

Библиотека UI-компонентов основана на shadcn/ui - коллекции переиспользуемых компонентов, построенных поверх Radix UI primitives. Проект включает более 50 компонентов: Button, Card, Dialog, Tabs, Table, Select, Input, Textarea, Badge, Alert, Accordion, Avatar, Breadcrumb, Calendar, Carousel, Checkbox, Collapsible, Command, ContextMenu, DropdownMenu, Form, HoverCard, Label, Menubar, NavigationMenu, Pagination, Popover, Progress, RadioGroup, Resizable, ScrollArea, Separator, Sheet, Sidebar, Slider, Sonner (toast), Switch, Toggle, Tooltip и другие. Все компоненты полностью типизированы, поддерживают тёмную тему и соответствуют стандартам доступности WAI-ARIA.

6.5. Стилизация и дизайн-система

Система стилизации основана на Tailwind CSS версии 4 с конфигурацией тёмной темы по умолчанию. Глобальные стили определены в `globals.css` и включают CSS-переменные для цветовой палитры (`background`, `foreground`, `card`, `primary`, `secondary`, `muted`, `accent`, `destructive`), скругления углов и радиусов. Шрифты Geist Sans и Geist Mono подключены через `next/font` в `layout.tsx`. Иконки предоставляются библиотекой Lucide Icons. Анимации и переходы реализованы через `framer-motion` для декларативного описания анимационных эффектов, включая появление этапов конвейера, переходы между вкладками и визуальная обратная связь при генерации.

7. Конфигурация пайплайна

Модуль `pipeline-config.ts` определяет конфигурацию конвейера обработки, включая типы продуктов, набор этапов для каждого типа и алгоритм автоматического определения типа продукта. Конфигурация построена на принципе адаптивности: различные типы продуктов требуют разных наборов UX-артефактов, и система автоматически

настраивает конвейер в соответствии с определённым типом. Это исключает генерацию нерелевантных артефактов (например, этап бронирования для SaaS-продукта) и оптимизирует использование ресурсов LLM.

7.1. Типы продуктов (ProductType)

Тип	Название	Характерные элементы
ecommerce	Электронная коммерция	Интернет-магазин, каталог, корзина, оплата
saas	SaaS-сервис	Подписка, дашборд, управление аккаунтом
b2b	B2B-платформа	Корпоративный портал, CRM, аналитика
blog	Блог / Контент	Статьи, комментарии, категории, авторы
landing	Лендинг	Целевая страница, СТА, форма захвата
dashboard	Дашборд / Аналитика	Графики, фильтры, экспорт данных
booking	Бронирование	Календарь, выбор услуги, подтверждение
app	Мобильное приложение	Навигация, профили, уведомления

7.2. Алгоритм detectProductType()

Функция detectProductType() реализует алгоритм оценки ключевых слов для автоматического определения типа продукта на основе текстового описания идеи. Алгоритм работает следующим образом: для каждого типа продукта определён набор ключевых слов и фраз с весовыми коэффициентами. Текст идеи нормализуется (приведение к нижнему регистру, удаление специальных символов) и проверяется на наличие каждого ключевого слова. При совпадении вес слова добавляется к суммарному баллу типа продукта. После обработки всех слов выбирается тип с максимальным суммарным баллом. Если ни один тип не набрал достаточный балл, используется тип по умолчанию (app). Алгоритм также поддерживает составные типы, когда описание содержит признаки нескольких категорий.

```
// pipeline-config.ts (фрагмент алгоритма)
function detectProductType(idea: string): ProductType {
  const text = idea.toLowerCase();
  const scores: Record<ProductType, number> = {};

  for (const [type, keywords] of Object.entries(PRODUCT_KEYWORDS)) {
    scores[type] = 0;
    for (const { word, weight } of keywords) {
      if (text.includes(word)) scores[type] += weight;
    }
  }

  const maxType = Object.entries(scores)
    .sort((a, b) => b[1] - a[1])[0];
  return maxType[1] > 0 ? maxType[0] : 'app';
}
```

7.3. Наборы этапов по типу продукта

Каждый тип продукта имеет собственный набор применимых этапов конвейера. Базовые этапы (idea, competitors, cjm, ia, userflow, prototype) являются обязательными для всех типов. Дополнительные этапы (invitation, guidelines, metrics) могут быть включены или исключены в зависимости от типа продукта. Например, тип "booking" включает этап invitation (приглашение к бронированию), а тип "landing" может исключить этап metrics, поскольку лендинг не требует сложной системы метрик. Конфигурация также определяет порядок выполнения этапов, который может отличаться для разных типов продуктов.

8. Развёртывание

Проект поддерживает три основных метода развёртывания: локальная разработка, развёртывание на платформе Vercel и развёртывание на собственном сервере. Каждый метод имеет свои особенности конфигурации и требования к окружению. Выбор метода зависит от целевого использования: локальная разработка для отладки и тестирования, Vercel для быстрого продакшен-развёртывания, собственный сервер для полного контроля над инфраструктурой.

8.1. Локальная разработка

```
# 1. Клонирование репозитория
git clone https://github.com/Wasp1977/intuiux_092026.git
cd intuiux_092026

# 2. Установка зависимостей
bun install

# 3. Создание конфигурации LLM API
cat > .z-ai-config << 'EOF'
{
  "baseUrl": "https://your-llm-api.com/v1",
  "apiKey": "your-api-key-here"
}
EOF

# 4. Инициализация базы данных
bunx prisma db push

# 5. Запуск сервера разработки
bun run dev
```

После выполнения этих команд приложение будет доступно по адресу <http://localhost:3000>. Сервер разработки Next.js поддерживает горячую перезагрузку, что позволяет видеть изменения в коде без перезапуска сервера. При первом запуске база данных SQLite будет создана автоматически в каталоге prisma/ на основе схемы schema.prisma.

8.2. Развёртывание на Vercel

```
# 1. Установка Vercel CLI
bun add -g vercel
```

```
# 2. Развёртывание в продакшен
vercel --prod

# 3. Настройка переменных окружения (через Vercel Dashboard)
# ZAI_BASE_URL = https://your-llm-api.com/v1
# ZAI_API_KEY = your-api-key-here
# DATABASE_URL = file:./dev.db
# GITHUB_TOKEN = ghp_xxxxx (опционально)
```

Vercel автоматически определяет Next.js-проект и настраивает сборку. Файл `vercel.json` содержит дополнительные настройки: максимальную длительность функции (`maxDuration`), регион развёртывания и правила маршрутизации. Важно отметить, что файловая система Vercel является эфемерной, поэтому база данных SQLite не сохраняется между развёртываниями. Для продакшена рекомендуется использовать внешний провайдер базы данных или убедиться, что база данных инициализируется при каждом развёртывании через `prisma db push`.

8.3. Развёртывание на собственном сервере

```
# 1. Сборка проекта
bun run build

# 2. Создание конфигурации LLM API
cat > .z-ai-config << 'EOF'
{
  "baseUrl": "https://your-llm-api.com/v1",
  "apiKey": "your-api-key-here"
}
EOF

# 3. Запуск в продакшен-режиме
NODE_ENV=production PORT=3000 bun run start
```

8.4. Caddyfile (обратный прокси)

```
# Caddyfile
your-domain.com {
  reverse_proxy localhost:3000
  encode gzip

  # Статические файлы с длинным кэшем
  handle /static/* {
    cache {
      ttl 365d
    }
  }

  # Автоматический HTTPS
  tls email@example.com
}
```

Caddy используется как обратный прокси для обеспечения HTTPS с автоматическим получением и обновлением сертификатов Let Encrypt, сжатия gzip и кэширования статических файлов. Конфигурация направляет все входящие запросы на порт 3000, где работает Next.js-сервер. Альтернативно можно использовать Nginx в качестве обратного прокси с ручной настройкой SSL-сертификатов.

8.5. Systemd-сервис

```
# /etc/systemd/system/intuiux-agent.service
[Unit]
Description=IntuiUX Agent (Next.js)
After=network.target

[Service]
Type=simple
User=www-data
WorkingDirectory=/opt/intuiux-agent
Environment=NODE_ENV=production
Environment=PORT=3000
ExecStart=/usr/local/bin/bun run start
Restart=on-failure
RestartSec=10
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target

# Управление сервисом
sudo systemctl daemon-reload
sudo systemctl enable intuiux-agent
sudo systemctl start intuiux-agent
sudo systemctl status intuiux-agent
```

9. Конфигурация Next.js и TypeScript

Конфигурация Next.js и TypeScript определяет параметры сборки, целевую среду выполнения и правила обработки исходного кода. Настройки оптимизированы для продакшен-развёртывания с учётом особенностей работы на платформе Vercel и использования Bun в качестве пакетного менеджера и рантайма.

9.1. next.config.ts

```
// next.config.ts
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  output: "standalone",
  typescript: {
    ignoreBuildErrors: true,
  },
  eslint: {
    ignoreDuringBuilds: true,
  },
  reactStrictMode: false,
};

export default nextConfig;
```

Ключевые параметры конфигурации: `output: "standalone"` создаёт автономную сборку, включающую все необходимые зависимости, что уменьшает размер развёртывания. `ignoreBuildErrors: true` отключает проверку типов TypeScript при сборке, что ускоряет процесс, но требует внимательности при разработке. `reactStrictMode: false` отключает

строгий режим React, что предотвращает двойной рендеринг в режиме разработки и уменьшает количество предупреждений в консоли.

9.2. tsconfig.json

```
// tsconfig.json
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "bundler",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "plugins": [{ "name": "next" }],
    "paths": { "@/*": [".src/*"] }
  },
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types"],
  "exclude": ["node_modules"]
}
```

Целевая версия JavaScript - ES2017, что обеспечивает широкую совместимость с браузерами. Module resolution установлен в "bundler", что корректно работает с Next.js и Bun. Путь "@/*" замарплен на ".src/*", позволяя использовать абсолютные импорты вида @/lib/zaи вместо относительных путей. Строгий режим TypeScript включён (strict: true), что обеспечивает максимальную типобезопасность при разработке.

9.3. Скрипты package.json

Скрипт	Команда	Описание
dev	next dev	Запуск сервера разработки с HMR
build	next build	Сборка продакшен-версии
start	next start	Запуск продакшен-сервера
lint	next lint	Проверка кода ESLint
db:generate	prisma generate	Генерация Prisma Client
db:push	prisma db push	Применение схемы к БД
db:migrate	prisma migrate dev	Создание миграций
db:reset	prisma db reset	Сброс базы данных

10. Система анализа конкурентов

Система анализа конкурентов является одной из ключевых подсистем IntuiUX Agent и состоит из трёх модулей, работающих совместно для обеспечения комплексного исследования конкурентного окружения. Модули реализуют конвейер обработки: от извлечения ключевых характеристик идеи до поиска реальных конкурентов и генерации структурированного анализа. Система включает резервную базу данных (fallback) для случаев, когда внешние API-поиска недоступны, что гарантирует работоспособность даже в офлайн-режиме или при достижении лимитов API.

10.1. idea-extractor.ts (v11.0)

Модуль idea-extractor.ts версии 11.0 отвечает за извлечение структурированной информации из текстового описания продуктовой идеи. Он использует LLM для анализа текста и выделения ключевых характеристик: название продукта, целевая аудитория, основная функциональность, уникальное ценностное предложение, отрасль и тип продукта. Модуль формирует специализированный промпт для роли "Product Analyst", который инструктирует LLM анализировать идею с точки зрения продуктового менеджмента и UX-исследований. Результат парсится как JSON и возвращается в виде структурированного объекта, который используется последующими этапами конвейера как контекст.

10.2. context-aware-competitor-search.ts

Модуль context-aware-competitor-search.ts обеспечивает контекстный поиск конкурентов с использованием внешних поисковых API. Модуль формирует поисковые запросы на основе извлечённых характеристик идеи: названия продукта, отраслевых ключевых слов и описания функциональности. Для каждого запроса выполняется вызов API поиска (при наличии SEARCH_API_KEY), результаты фильтруются и ранжируются по релевантности. Модуль поддерживает несколько поисковых стратегий: точный поиск по названию, поиск по отрасли и функциональности, поиск по целевой аудитории. При недоступности API поиска модуль автоматически переключается на резервную базу данных из competitor-analyzer.ts.

10.3. competitor-analyzer.ts (v3.0)

Модуль competitor-analyzer.ts версии 3.0 выполняет глубокий анализ конкурентов с использованием LLM. Модуль получает список найденных конкурентов и формирует промпт, инструктирующий LLM выполнить сравнительный анализ: выявить сильные и слабые стороны каждого конкурента, определить общие паттерны UX, найти возможности для дифференциации и сформулировать рекомендации по позиционированию. Результат возвращается в формате JSON с детальным профилем каждого конкурента и сводной матрицей сравнения.

10.4. Резервная база данных

Система включает резервную базу данных конкурентов для пяти основных отраслей, которая используется при недоступности внешних API поиска. База содержит предварительно собранную информацию о типичных конкурентах в каждой отрасли с описанием их функциональности, целевой аудитории и UX-характеристик.

Отрасль	Название (рус.)	Примеры конкурентов
telecom	Телекоммуникации	VoIP-сервисы, облачные АТС, мессенджеры
dashboards	Дашборды и аналитика	Grafana, Tableau, Metabase, PowerBI
ecommerce	Электронная коммерция	Shopify, WooCommerce, BigCommerce
fitness	Фитнес и здоровье	MyFitnessPal, Strava, Fitbit
education	Образование	Coursera, Udemy, Khan Academy

11. Генерация артефактов

Каждый этап конвейера генерирует определённый тип UX-артефакта. Генерация осуществляется через LLM с использованием специализированных промптов для каждого этапа. Результаты парсятся, валидируются и сохраняются в базу данных. Некоторые артефакты (CJM, IA, Userflow) визуализируются через Mermaid.js, а прототипы генерируются как полноценные HTML-SPA с тёмной темой и интерактивными элементами.

11.1. Этап Idea (Анализ идеи)

Этап Idea является первым шагом конвейера и выполняет формализацию продуктовой идеи. Промпт для этого этапа описывает роль "Product Analyst" и инструктирует LLM проанализировать текст идеи, выделив следующие компоненты: название продукта, описание (2-3 предложения), целевая аудитория с демографическими характеристиками, основные функции (4-6 пунктов), уникальное ценностное предложение, тип продукта из предопределённого списка, отрасль и конкуренты. LLM возвращает результат в формате JSON, который парсится и используется как контекст для всех последующих этапов конвейера.

```
// Пример результата этапа Idea
{
  "productName": "CloudPBX",
  "description": "Облачная АТС для малого бизнеса...",
  "targetAudience": "Малый бизнес, 5-50 сотрудников...",
  "features": ["Маршрутизация вызовов", "IVR меню", "..."],
  "valueProposition": "Полнофункциональная АТС без оборудования",
  "productType": "saas",
  "industry": "telecom",
  "competitors": ["RingCentral", "8x8", "Zoom Phone"]
}
```

11.2. Этап CJM (Карта пути клиента)

Этап CJM генерирует карту пути клиента (Customer Journey Map) в формате Mermaid journey-диаграммы. Промпт инструктирует LLM описать полный путь клиента от первого контакта с продуктом до достижения основной цели, включая этапы осознания,

рассмотрения, покупки, использования и поддержки. Для каждого этапа указываются действия клиента, точки касания (touchpoints), эмоции (от 1 до 5) и потенциальные проблемы. Результат представляется в синтаксисе Mermaid journey, который рендерится компонентом MermaidRenderer.tsx в визуальную диаграмму с цветовой кодировкой эмоций.

```
journey
  title Путь клиента облачной АТС
  section Осознание
    Поиск решения: 3: Пользователь
    Чтение статей: 4: Пользователь
  section Рассмотрение
    Сравнение вариантов: 3: Пользователь
    Запрос демо: 4: Пользователь
  section Покупка
    Выбор тарифа: 4: Пользователь
    Оплата: 3: Пользователь
  section Использование
    Настройка АТС: 2: Пользователь
    Подключение номеров: 3: Пользователь
```

11.3. Этап Prototype (Прототип)

Этап Prototype генерирует полноценный интерактивный HTML-прототип в виде Single Page Application (SPA). Промпт инструктирует LLM создать самодостаточный HTML-файл со встроенными CSS и JavaScript, который визуально представляет ключевые экраны продукта. Прототип использует тёмную тему дизайна с цветовой палитрой, согласованной с основным приложением IntuiUX Agent. HTML-прототип включает навигацию между экранами, интерактивные элементы (кнопки, формы, модальные окна) и адаптивную вёрстку. Результат сохраняется в каталоге prototypes/ и доступен через API-маршрут /api/prototype/[slug].

11.4. Mermaid-диаграммы

Несколько этапов конвейера генерируют артефакты в формате Mermaid для визуализации. Этап CJM использует тип диаграммы "journey" для отображения карты пути клиента с эмоциональной шкалой. Этап IA (Информационная архитектура) генерирует два типа диаграмм: "flowchart" для структуры навигации и "er" для ER-диаграммы сущностей базы данных. Этап Userflow использует "flowchart" для визуализации пошагового потока взаимодействия пользователя с продуктом, включая ветвления и альтернативные пути. Все диаграммы рендерятся на клиенте через MermaidRenderer.tsx с тёмной темой и поддержкой масштабирования.

Этап	Тип Mermaid	Содержание
CJM	journey	Карта пути клиента с эмоциями
IA (навигация)	flowchart	Структура меню и навигации
IA (данные)	er	ER-диаграмма сущностей БД
Userflow	flowchart	Поток взаимодействия пользователя

12. Безопасность

Безопасность является важным аспектом архитектуры IntuiUX Agent. Система реализует несколько уровней защиты, включая изоляцию конфигурационных данных, аутентификацию API-запросов, песочницу для рендеринга пользовательского контента и обработку ошибок. Хотя проект не является финансовым или медицинским приложением, принятые меры обеспечивают достаточный уровень защиты для инструмента UX-исследований.

12.1. Защита учётных данных

Файл `.z-ai-config`, содержащий `baseUrl` и `apiKey` LLM-провайдера, включён в `.gitignore` для предотвращения случайной фиксации учётных данных в системе контроля версий. При развёртывании на Vercel учётные данные передаются через переменные окружения, которые хранятся в зашифрованном виде на платформе. Рекомендуется использовать отдельные ключи API для разработки и продакшена, а также регулярно ротировать ключи в соответствии с политиками безопасности организации.

12.2. Аутентификация API-запросов

Все запросы к LLM API выполняются с заголовком `X-Token`, содержащим ключ аутентификации. Этот заголовок добавляется автоматически классом LLM в модуле `zai.ts`. Ключ не логируется и не включается в ответы API. Для дополнительной защиты рекомендуется ограничить IP-адреса, с которых допустимы запросы к API, на стороне провайдера LLM.

12.3. Изоляция контента (iframe sandbox)

Сгенерированные HTML-прототипы отображаются в `iframe` с атрибутом `sandbox`, что обеспечивает строгую изоляцию от основного приложения. Атрибут `sandbox` ограничивает возможности встроеного документа: запрещает выполнение скриптов из чужих источников, предотвращает доступ к родительскому контексту, блокирует всплывающие окна и отправку форм. Это критически важно для безопасности, поскольку содержание прототипов генерируется LLM и может содержать потенциально опасный код. Изоляция гарантирует, что даже при наличии вредоносного кода в прототипе, он не сможет повлиять на основное приложение или получить доступ к данным пользователя.

12.4. Обработка ошибок

```
// Паттерн обработки ошибок в API-маршрутах
try {
  const result = await llm.chat(messages, options);
  const parsed = JSON.parse(result);
  await prisma.pipelineStage.update({
    where: { id: stageId },
    data: { status: 'completed', result: parsed, completedAt: new Date() }
  });
} catch (error) {
  await prisma.pipelineStage.update({
    where: { id: stageId },
```

```
    data: { status: 'error', error: String(error) }
  });
  return NextResponse.json({ error: 'Generation failed' }, { status: 500 });
}
```

12.5. Остановка генерации

Механизм `stop_generation` позволяет пользователю прервать текущий процесс генерации на любом этапе. При вызове операции `stop_generation` система устанавливает флаг остановки в `PipelineSession`, который проверяется перед началом каждого следующего этапа. Текущий этап завершается (его результат сохраняется), но последующие этапы пропускаются. Это предотвращает ненужные вызовы LLM API и позволяет экономить ресурсы при ошибочном запуске конвейера.

13. Устранение неполадок

В данном разделе собраны наиболее распространённые проблемы, с которыми сталкиваются разработчики при установке и развёртывании IntuiUX Agent, а также методы их решения. Для каждой проблемы указаны симптомы, вероятная причина и рекомендуемое действие.

Проблема	Вероятная причина	Решение
API возвращает 401	Неверный или отсутствующий API-ключ	Проверить <code>.z-ai-config</code> и <code>ZAI_API_KEY</code>
Пустой ответ LLM	Превышен лимит токенов или таймаут	Уменьшить <code>max_tokens</code> , проверить сеть
Mermaid не рендерится	Ошибка синтаксиса диаграммы	Проверить Mermaid-код в консоли браузера
БД не найдена	<code>DATABASE_URL</code> указывает на несуществующий файл	Выполнить <code>bunx prisma db push</code>
Прототип не отображается	<code>iframe sandbox</code> блокирует скрипты	Проверить атрибут <code>sandbox</code> <code>iframe</code>
Vercel: ошибка сборки	<code>ignoreBuildErrors</code> скрывает ошибки TS	Запустить <code>npx tsc --noEmit</code> локально
Генерация зависает	LLM-провайдер недоступен	Проверить доступность <code>baseUrl</code> , использовать <code>stop_generation</code>
Конкуренты не найдены	<code>SEARCH_API_KEY</code> не установлен	Установить ключ или использовать <code>fallback</code> -данные

13.1. Логирование

Приложение использует стандартный механизм логирования Next.js через `console.log`, `console.warn` и `console.error`. В продакшен-режиме логи направляются в `stdout/stderr` и доступны через систему журналирования платформы развёртывания (Vercel Logs,

journalctl для systemd). API-маршруты логируют ключевые события: начало и завершение обработки этапа конвейера, ошибки при вызове LLM, проблемы с подключением к БД. Чувствительные данные (API-ключи, промпты целиком) не логируются.

13.2. Команды проверки конфигурации

```
# Проверка доступности LLM API
curl -s -o /dev/null -w "%{http_code}" \
  -H "X-Token: $ZAI_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"model":"default","messages":[{"role":"user","content":"test"}]}' \
  $ZAI_BASE_URL/chat/completions

# Проверка файла конфигурации
cat .z-ai-config | python3 -m json.tool

# Проверка базы данных
bunx prisma db pull --print

# Проверка переменных окружения
echo "ZAI_BASE_URL=$ZAI_BASE_URL"
echo "DATABASE_URL=$DATABASE_URL"

# Проверка порта сервера
curl -s -o /dev/null -w "%{http_code}" http://localhost:3000

# Проверка версии Bun
bun --version

# Проверка версии Node.js (если используется)
node --version
```

14. Расширение проекта

Архитектура IntuiUX Agent спроектирована с учётом расширяемости. Разработчики могут добавлять новые этапы конвейера, новые типы продуктов и заменять LLM-провайдера без значительных изменений существующего кода. В данном разделе описаны пошаговые инструкции для каждого типа расширения.

14.1. Добавление нового этапа конвейера

Для добавления нового этапа конвейера необходимо выполнить следующие шаги, которые затрагивают несколько файлов проекта. Новый этап должен следовать единому паттерну обработки, принятому в существующих этапах, и корректно интегрироваться с системой управления зависимостями между этапами.

```
# Шаг 1: Создать API-маршрут
# src/app/api/pipeline/stages/newstage/route.ts
export async function POST(req: Request) {
  const { sessionId } = await req.json();
  // ... стандартный паттерн обработки этапа
}

# Шаг 2: Зарегистрировать этап в pipeline-config.ts
# Добавить имя этапа в массивы stages для нужных ProductType
```

```
# Шаг 3: Добавить модель в Prisma (если нужны спец. поля)
# Обновить prisma/schema.prisma при необходимости

# Шаг 4: Добавить UI для отображения результата
# Обновить page.tsx: добавить вкладку/секцию для нового этапа

# Шаг 5: Протестировать
bun run dev
# Отправить идею и проверить, что новый этап выполняется
```

14.2. Добавление нового типа продукта

Для добавления нового типа продукта необходимо обновить несколько конфигурационных файлов. Тип продукта определяет набор этапов конвейера и ключевые слова для автоматического определения типа. Новый тип должен иметь уникальный идентификатор, описание, набор ключевых слов с весами и список применимых этапов конвейера.

```
# Шаг 1: Определить тип в pipeline-config.ts
// Добавить в enum/union ProductType:
type ProductType = 'ecommerce' | 'saas' | ... | 'newtype';

# Шаг 2: Добавить ключевые слова для detectProductType()
const PRODUCT_KEYWORDS = {
  newtype: [
    { word: 'ключевое_слово_1', weight: 10 },
    { word: 'ключевое_слово_2', weight: 8 },
    { word: 'синоним', weight: 5 },
  ],
  // ...
};

# Шаг 3: Определить этапы для нового типа
const PRODUCT_STAGES = {
  newtype: ['idea', 'competitors', 'cjm', 'ia', 'userflow', 'prototype'],
  // ...
};

# Шаг 4: Протестировать определение типа
# Отправить идею с ключевыми словами нового типа
# Проверить, что detect_product_type вернет корректный тип
```

14.3. Замена LLM-провайдера

Система спроектирована так, чтобы замена LLM-провайдера требовала минимальных изменений. Класс LLM в модуле `zai.ts` абстрагирует взаимодействие с API через стандартный интерфейс OpenAI-совместимых эндпоинтов (POST `/chat/completions`). Для замены провайдера достаточно обновить `baseUrl` и `apiKey` в файле `.z-ai-config` или переменных окружения. Если новый провайдер использует нестандартный формат API, может потребоваться адаптация метода `chat()` в `zai.ts` - например, изменение формата заголовка аутентификации или структуры тела запроса. Рекомендуется сохранить существующий интерфейс метода `chat()`, чтобы не вносить изменения в вызовы по всему проекту.

```
# Замена LLM-провайдера

# Вариант 1: Через .z-ai-config
cat > .z-ai-config << 'EOF'
{
  "baseUrl": "https://new-provider.com/v1",
  "apiKey": "new-api-key"
}
EOF

# Вариант 2: Через переменные окружения
export ZAI_BASE_URL="https://new-provider.com/v1"
export ZAI_API_KEY="new-api-key"

# Вариант 3: На Vercel (Dashboard -> Settings -> Environment Variables)
# ZAI_BASE_URL = https://new-provider.com/v1
# ZAI_API_KEY = new-api-key

# Проверка работоспособности нового провайдера
curl -X POST "$ZAI_BASE_URL/chat/completions" \
  -H "Content-Type: application/json" \
  -H "X-Token: $ZAI_API_KEY" \
  -d '{"model": "default", "messages": [{"role": "user", "content": "test"}]}'
```